

Hybrid RAG System - Working Explanation

Overview

This project implements a **Hybrid Retrieval-Augmented Generation (Hybrid RAG)** system that combines:

- **FAISS Vector Search (Dense Retrieval)** for semantic understanding
- **BM25 Keyword Search (Sparse Retrieval)** for exact keyword matching
- **LangGraph Self-Correcting Workflow** for retrieval validation and query rewriting
- **Groq LLM** for grounded answer generation

The goal is to improve retrieval quality compared to traditional vector-only RAG systems.

Complete Pipeline Flow

```
Documents
↓
Load Documents
↓
Chunk Documents
↓
Generate Embeddings
↓
Create FAISS Index
↓
Create BM25 Index
↓
User Question
↓
Hybrid Retrieval
(FAISS + BM25)
↓
Relevance Grading
↓
Query Rewrite (if needed)
↓
LLM Answer Generation
↓
Answer + Sources
```

Step 1: Document Ingestion

The system loads documents from the `data/` folder.

Supported formats:

- PDF
- DOCX
- CSV
- Excel
- TXT
- Images (OCR)

Each document is converted into a LangChain Document object.

Example:

```
{
  "page_content": "...",
  "metadata": {
    "source": "BlackBook.pdf",
    "file_type": "pdf"
  }
}
```

The metadata is preserved and later used for source attribution.

Step 2: Text Chunking

Large documents are split into smaller chunks before embedding.

Configuration:

```
Chunk Size: 1000 characters
Chunk Overlap: 150 characters
```

Example:

```
Original Document
-----
3000 characters
```

After Chunking

Chunk 1
0 - 1000 chars

Chunk 2
850 - 1850 chars

Chunk 3
1700 - 2700 chars

The overlap ensures information isn't lost when content spans chunk boundaries.

Step 3: Embedding Generation

The embedding model converts chunks into vectors.

Model:

all-MiniLM-L6-v2

Process:

Text Chunk
↓
Embedding Model
↓
384-Dimensional Vector

Example:

"PSK modulation"
↓
[0.12, -0.44, 0.88, ...]

The embedding captures semantic meaning.

Because of this:

PSK modulation

and

Phase Shift Keying

produce similar vectors even though the wording differs.

Step 4: Dual Indexing

This is the foundation of Hybrid RAG.

The project creates two independent retrieval systems.

A. FAISS Index (Dense Retrieval)

Stores embeddings.

Workflow:

```
Question
↓
Embedding
↓
Nearest Vector Search
↓
Top Matching Chunks
```

Best for:

- Semantic search
- Synonyms
- Paraphrases
- Meaning-based retrieval

Example:

Question:

How is data encoded?

Retrieved Chunk:

Phase Shift Keying changes the carrier phase...

Even though the exact words differ.

B. BM25 Index (Sparse Retrieval)

Stores keyword statistics.

Workflow:

```
Question
  ↓
Keyword Matching
  ↓
Top Matching Chunks
```

Best for:

- Exact terms
- Acronyms
- Technical keywords
- Names

Example:

```
OFDM
PSK
QPSK
```

BM25 performs better than embeddings for these exact matches.

Step 5: Hybrid Retrieval

Question:

What is PSK modulation?

FAISS Results

Chunk A
Phase Shift Keying...

Score 0.82

Chunk B
Digital modulation...

Score 0.79

BM25 Results

Chunk C
PSK modulation...

Score 0.91

Chunk D
PSK table...

Score 0.73

Fusion

Weights:

FAISS = 0.6

BM25 = 0.4

Formula:

Final Score =
(0.6 × Semantic Score)
+
(0.4 × Keyword Score)

The system then:

1. Merges results
2. Removes duplicates
3. Sorts by score
4. Selects top chunks

Result:

Top 4 Chunks

This gives better retrieval than either method alone.

Step 6: LangGraph Self-Correcting Workflow

Traditional RAG:

```
Retrieve
↓
Generate
```

This project:

```
Retrieve
↓
Grade Relevance
↓
Relevant?
├─ Yes → Generate
└─ No
    ↓
    Rewrite Query
    ↓
    Retrieve Again
```

Example

User Query:

What is PSK modulation?

Typo:

moduation

First Retrieval

Retrieved Chunks:

General methodology documents

Grader:

Not Relevant

Query Rewrite

LangGraph rewrites:

PSK modulation

to

Phase Shift Keying modulation

Second Retrieval

Now retrieval finds:

Actual PSK chunks

Grader:

Relevant

Answer generation proceeds.

This significantly improves robustness against:

- Typos
- Poor phrasing
- Ambiguous queries

Step 7: Answer Generation

The final chunks are inserted into a RAG prompt.

Example:

Context:

Chunk 1:

PSK is a digital modulation technique...

Chunk 2:

QPSK uses four phase states...

Question:

What is PSK?

Prompt Rules:

Use only context.
Do not hallucinate.
Cite sources.

LLM:

openai/gpt-oss-20b (Groq)

generates the final response.

End-to-End Example

Suppose the document contains:

PSK
QPSK
16-QAM
OFDM

User asks:

What modulation techniques are discussed?

Retrieval Phase

FAISS finds:

PSK
OFDM

because they are semantically related.

BM25 finds:

Modulation Techniques Table

because of exact keyword matches.

Fusion Phase

The results are combined and ranked.

Top chunks are selected.

Relevance Check

Are the retrieved chunks relevant?

Result:

YES

Answer Generation

The LLM receives:

PSK...
QPSK...
16-QAM...
OFDM...

Final Answer

The documents discuss:

1. PSK
2. QPSK
3. 16-QAM
4. OFDM

Source:
BlackBook.pdf

Why This Is Better Than Standard RAG

Vector-Only RAG

Strengths:

- Semantic understanding

Weaknesses:

- Acronyms
- Exact keywords
- Technical terms

Example:

Query :
OFDM

May not retrieve the best chunk.

Hybrid RAG

Uses:

FAISS
+
BM25

Benefits:

- Exact keyword matching
 - Semantic understanding
 - Better recall
 - Better precision
-

Current Strengths

Implemented:

- FAISS Semantic Retrieval
- BM25 Keyword Retrieval
- Hybrid Score Fusion
- LangGraph Query Rewriting
- Relevance Grading
- Source Attribution
- Persistent FAISS Index
- Multi-Format Document Support
- GPU Support for Embeddings

Current Limitations

Not yet implemented:

- Cross Encoder Reranking
- Reciprocal Rank Fusion (RRF)
- Context Compression
- Metadata Filtering
- Multi-Hop Retrieval
- RAG Evaluation Framework

Assessment

Area	Score
Basic RAG	9/10
Hybrid Retrieval	8.5/10
LangGraph Workflow	8/10
Production Readiness	6.5/10
Resume Value	8.5/10

Recommended Next Improvements

High Priority

1. Cross Encoder Reranking
2. Reciprocal Rank Fusion (RRF)
3. Metadata Filters

Medium Priority

1. Streaming Responses
2. Context Compression
3. RAG Evaluation (RAGAS)

Advanced

1. Multi-Hop Retrieval

2. Graph RAG
3. Agentic RAG
4. Long-Term Memory